Coffee White Stem Borer Detection Model

How to build it from scratch: a step-by-step guide from field photos to a working phone app

This guide is for	Field collectors, the agronomist / entomologist, the dataset lead, and the ML / app developers.
Based on	An image-by-image audit of the team's Oct–Nov 2026 field photos, the 2019 ISPRS paper (Bhandarkar et al.), and a review of the earlier ChatGPT handbook.
Version	1.0 · 11 November 2026
Status of the model	Not built yet. Today there are 25 proven infected photos (about 6 plants) and 0 proven healthy photos. Collecting data is the first job.

How to read this guide
Part A explains **what** we are building and **why** in plain language. Part B is the **step-by-step build**, in the exact order to do it. Every phase ends with a **gate**: a short checklist that must pass before moving on. Part C has templates, checklists and a glossary.
Code is written in Python (PyTorch). Everything else assumes no machine-learning background.

Contents

Part A: Understanding the problem

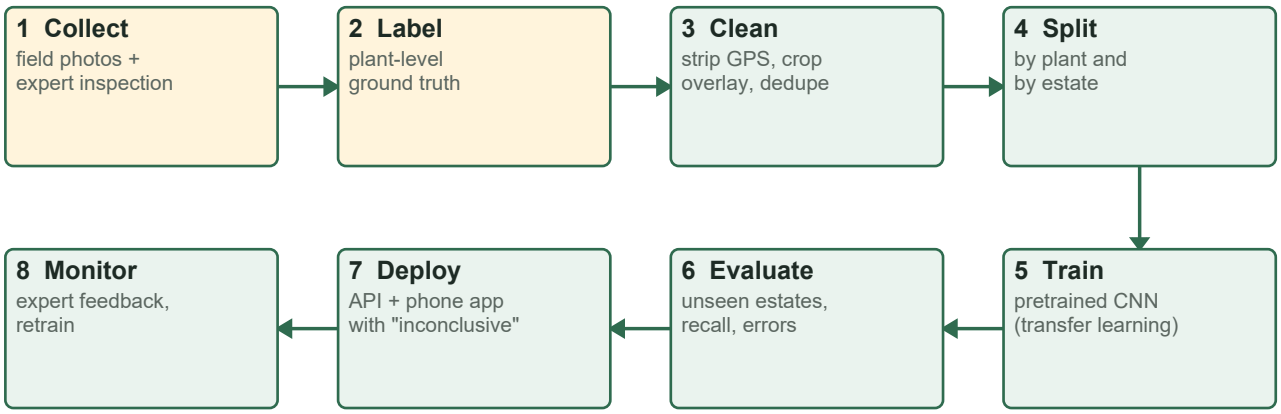
1. Where we are today
2. What exactly we are building
3. Machine-learning ideas you need (in plain words)

Part B: Building it, step by step

4. Phase 0: Team, tools and decisions
5. Phase 1: Field data collection (the most important phase)
6. Phase 2: Labelling and data management
7. Phase 3: Cleaning and preparing images
8. Phase 4: Splitting the data without cheating
9. Phase 5: Training the model
10. Phase 6: Evaluation: proving it works
11. Phase 7: Improving the model
12. Phase 8: Deployment: API and phone app
13. Phase 9: Monitoring and retraining
14. Optional: adding the environmental and sensor data

Part C: Reference

15. Timeline, gates and roles
16. Risks and how to handle them
17. Templates and checklists
18. What we kept from the paper and the ChatGPT plan, and what we changed
19. Glossary



Amber steps (1-2) are where this project is today, and they decide whether everything after them works.

1. Where we are today

Before planning anything, here is an honest picture of what the team has, based on looking at every photo one by one.

Item	Finding
Photos received	338 files. 32 "tab images" are copies of iPad photos already in "drive images", so **305 are unique**.
Field trip	28 Oct – 2 Nov 2026, 6 sites: Kushalnagar, Ponnampet, Somwarpet (Kodagu); Kundur/Koppa; Betta Damalali; CCRI Balehonnur.
Proven infected photos	**25** (about 6 plants). 19 are split stems showing larval tunnels, grubs or beetles; 6 show an exit hole on the stem.
Probably infected	27 photos of plants recorded as infested on field sheets, but the photo itself only shows ridges or dieback.
Proven healthy photos	**0.** No photo shows a clean, clearly visible main stem on a plant inspected and found free of WSB.
Labels	Exist only on paper field sheets. Photos were not linked to plant IDs, so labels had to be reconstructed from timestamps.
Hidden problems found	A GPS-overlay app burned location text into 148 photos; most "healthy" photos came from one site; the 2019 paper's numbers do not add up.

The single most important conclusion
The bottleneck is not the AI. It is **verified photographs of healthy and infected stems, taken the same way, at the same estates, linked to an expert diagnosis**. A small model trained on good data beats a big model trained on unclear data. That is why Phase 1 (field collection) is the longest section of this guide.

What the existing data is still good for

- **The 25 confirmed images:** a reference gallery showing annotators and the agronomist what confirmed WSB damage looks like. Most are split stems (an inside view), which a farmer will never photograph, so they are **proof**, not training data for the field model.
- **The 6 exit-hole photos and 27 "probable" photos:** external stem views. After an expert signs off on them, they can join the positive class.
- **Field sheets:** the right protocol is already in place (ridges, exit holes, frass, wilting, sun exposure, sensors). It just needs plant IDs linked to photos.
- **Lessons learned:** every mistake found in the audit (below) is turned into a rule in this guide.

Mistake found in the audit	Rule in this guide
Photos not linked to plant IDs	Photograph a plant-ID card first, every time (Phase 1)
GPS Map Camera text burned into pictures	Use the phone's normal camera app only (Phase 1)
"Healthy" assumed from a site note	Healthy only after an expert inspects that plant's stem (Phases 1-2)
Healthy and infected from different sites and phones	Collect both classes at every estate, with every phone (Phase 1)
Same plant photographed 10+ times	Count plants, not photos; split by plant (Phase 4)
Mossy or blurred stems	Quality checklist at capture time; keep mossy healthy stems on purpose (Phase 1)

2. What exactly we are building

The product, in one sentence

A farmer photographs the **main stem** of an Arabica coffee plant (plus a whole-plant photo), and the app answers with one of three messages:

App answer	Meaning	What the farmer is told
WSB signs likely	The stem looks like the expert-confirmed infected stems the model learned from	Inspect this plant; contact an extension officer / expert to confirm
No visible WSB signs	The stem looks like the expert-verified healthy stems	No action needed now; keep monitoring, especially in flight season
Inconclusive	Photo too blurry/dark, stem not visible, or the model is unsure	Retake the photo following the on-screen guide, or ask an expert

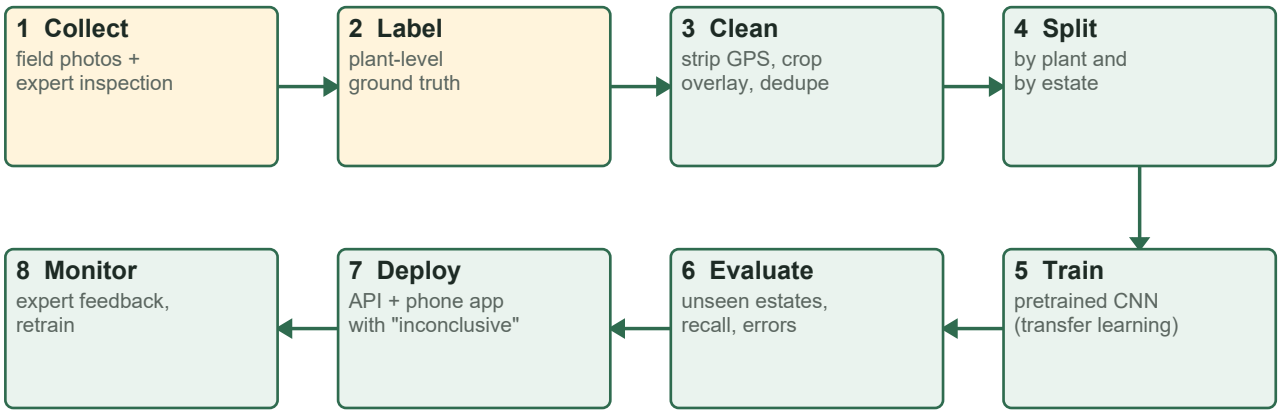
What Version 1 will NOT do (and why)

- **It will not see larvae hidden inside a healthy-looking stem.** A camera only sees the surface. Early infestation can be invisible; the app must say "no *visible* signs", never "no WSB".
- **It will not diagnose from leaves alone.** Yellow leaves have many causes (rust, drought, nutrients). WSB evidence is on the stem.
- **It will not tell farmers to uproot or spray.** The output is advice to inspect, not a treatment order.
- **It will not forecast beetle emergence from weather.** That is a separate, later model (Section 14).

Why stems, and not whole-plant photos like the 2019 paper

The 2019 paper classified whole-plant images and reported about 85% accuracy, but its confusion matrix actually gives 70%, and its splits probably mixed photos of the same plant between training and testing. Whole-plant photos mostly show leaves and background, so a model learns the estate, the light and the camera rather than the pest. The pest leaves its marks on the stem: **ridges, oval exit holes, frass, cracked or ringed bark**. Making the stem the main photo points the model at the real evidence.

The full system at a glance



Amber steps (1-2) are where this project is today, and they decide whether everything after them works.

Each box is one phase in Part B. Phases 1-4 are about data and take most of the time. Phase 5 (training) is short once the data is right.

3. Machine-learning ideas you need (in plain words)

What a model is

An image model is a program that turns a photo into a number between 0 and 1, here the "WSB score". It learns by looking at many photos whose correct answers are already known, and adjusting itself until its scores match those answers. It does not know biology. It only learns visual patterns that separate the two groups of photos *you give it*. If your infected photos are all from Kodagu and your healthy photos all from Balehonnur, it will happily learn "Kodagu vs Balehonnur".

Transfer learning: why we do not start from zero

Training an image model from nothing needs millions of photos. Instead we take a model already trained on millions of everyday images (ImageNet and similar), which already "sees" edges, textures and shapes, and we only teach its last layers our two classes. This is called **transfer learning**, and it is what makes a few hundred plants enough to start.

Plant vs photo: the most common trap

Ten photos of one plant teach the model about **one** plant. What matters is how many **different plants** each class has, across how many **estates**, **phones** and **lighting conditions**. Always count plants.

Training, validation and test sets

Set	What it is used for	Analogy
Training (about 70% of plants)	The model learns from these	Textbook and practice problems
Validation (about 15%)	Choose settings and the decision threshold	Mock exam
Test (about 15%, plus a whole held-out estate)	Measured once, at the very end	The real exam, never seen before

If any photo of a plant appears in two sets, the model can recognise the plant instead of the pest, and the score is inflated. This is called **data leakage**.

The four outcomes, and the two numbers that matter most

	Model says WSB	Model says healthy
Really infected	True positive (TP): caught	**False negative (FN): missed. The worst error**
Really healthy	False positive (FP): false alarm	True negative (TN): correct

- **Recall (sensitivity)** = TP / (TP + FN): of all truly infected plants, how many did we catch? **Our main target.**
- **Precision** = TP / (TP + FP): of all alerts, how many were real? Keeps farmers from losing trust.
- **Accuracy alone is misleading.** If 90% of plants are healthy, a model that always says "healthy" scores 90% and is useless.

Augmentation

Training-time changes to photos (small rotations, crops, brightness changes) that teach the model a stem is still the same stem in different light or angles. Used only on training photos, never on test photos.

Threshold and "inconclusive"

The model gives a score from 0 to 1. We choose two cut-offs on the validation set, for example: score above 0.6 means "WSB likely", below 0.3 means "no visible signs", and anything in between means "inconclusive". The cut-offs are chosen to keep recall high, not set by guesswork.

4. Phase 0: Team, tools and decisions

4.1 People (one person can hold several roles)

Role	Responsible for	Time needed
Project owner	Scope, budget, approvals at each gate	A few hours/week
Agronomist / entomologist	Defines what counts as infected/healthy; inspects and signs off plants; reviews model errors	1-2 days/week during collection
Field lead + 1-2 collectors	Visits estates, tags plants, takes photos, fills sheets	Full time during collection
Dataset lead	Ingests photos daily, checks quality, keeps metadata, links photos to plants	2-3 days/week
ML developer	Cleaning scripts, training, evaluation	Part time early, full time from Phase 5
App / backend developer	API, phone app, storage	From Phase 8

4.2 Decisions to write down before collecting

1. **Target user and photo type:** V1 = a farmer or field officer photographing the main stem from about 50-70 cm, plus a whole-plant photo.
2. **Class definitions:** use exactly the definitions in Section 6.1. The agronomist signs them.
3. **Estates:** list at least 6 estates (more is better), covering Kodagu, Chikkamagaluru and Hassan areas, sun and shade, old and young plants.
4. **Phones:** at least 3 different models (include a cheap Android, since farmers will use those). Use the same phones for both classes.
5. **Data ownership and consent:** written permission from each estate to photograph and use images for model training.
6. **Success targets** (to confirm after the baseline): e.g. recall of 90% or more on confirmed infected plants from unseen estates, with precision of 70% or more.

4.3 Tools

Tool	Use	Cost
Phones (normal camera app)	Photos	Existing
Printed plant-ID cards / slate + marker	Put plant ID in the first photo of each plant	Minimal
Tree tags or numbered ribbons	Physically mark inspected plants for revisits	Minimal
Google Drive / shared drive + daily upload	Raw photo storage (original files, never edited)	Free / existing
Spreadsheet (Google Sheets / Excel)	plants.csv and images.csv (Section 17)	Free
Python 3.12 + VS Code	All scripts	Free
PyTorch + timm + scikit-learn	Training and evaluation	Free
Google Colab or Kaggle Notebooks (GPU)	Training if the laptop has no GPU	Free tier is enough at first
CVAT or Label Studio (optional)	Drawing boxes around stems/holes later	Free
FastAPI + Docker (later)	Prediction service for the app	Free; server about Rs 1-3k/month

Environment setup

```
# One-time setup (Windows PowerShell, inside the project folder)
py -3.12 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install torch torchvision timm scikit-learn pandas numpy opencv-python pillow pillow-heif matplotlib tqdm
pip install grad-cam fastapi uvicorn onnx onnxruntime        # needed later (Phases 6-8)
```

Gate 0: pass before going to the field

[ ] Roles assigned and agronomist available on collection days
[ ] Class definitions (Section 6.1) signed by the agronomist
[ ] At least 6 estates confirmed, with written permission
[ ] Phones chosen; GPS Map Camera (and any overlay/beauty apps) uninstalled or not used
[ ] Plant-ID cards, tags, printed field sheets ready; shared drive folder created

5. Phase 1: Field data collection

Timing matters: start now
The winter flight period is **October to December** and the summer flight is **April to May**. Fresh exit holes and active damage are easiest to find during and just after these periods. Collecting in November–December 2026 gives the best chance of clear positives.

5.1 How many plants?

There is no exact formula; the learning curve in Phase 7 will tell us if more is needed. Use these working targets, counting **plants**, not photos:

Stage	Infected plants	Healthy plants	Other-stress plants	Estates
First baseline (proof of concept)	100 or more	100 or more	30 or more	4 or more
Pilot model (field test)	300 or more	300 or more	100 or more	6 or more
Production	1,000 or more, growing over time	Same	300 or more	10 or more

With 3-5 photos per plant, 300 + 300 plants means roughly 2,000-3,000 photos. A two-person team photographing and inspecting carefully can do about 40-60 plants per day.

5.2 Choose plants the right way

- **At every estate, collect both classes.** For each infected plant found, also record 1-2 healthy plants nearby (same row/block, same light). This is the single best defence against the model learning "site" instead of "pest".
- **Use every phone for both classes.** Rotate phones between collectors each day.
- **Include hard healthy cases on purpose:** moss- or lichen-covered stems, lime-swabbed stems, old gnarled stems, naturally rough bark. If all healthy stems are smooth and all infected ones rough, the model learns "rough = infected".
- **Include other-stress plants:** leaf rust, drought wilting, nutrient yellowing, mechanical damage, other borers. These teach the model what WSB is *not*.
- **Include different severities:** early (ridges only), medium (ridges + few holes), advanced (many holes, dieback).

5.3 How each plant is diagnosed (ground truth)

Label	Requirements (all must be true)
INFECTED_CONFIRMED	Expert sees fresh or old **exit holes** or **larval tunnels/frass**, OR the stem was split (only plants already marked for uprooting) and tunnels/grubs were found.
INFECTED_PROBABLE	Expert sees typical **ridges** or girdling and judges WSB likely, but no hole or tunnel is seen.
HEALTHY_VERIFIED	Expert inspects the **whole main stem circumference from ground level to about 1 m** and the main primaries: no ridges, no exit holes, no frass, no wilting/yellowing typical of WSB.
OTHER_STRESS	HEALTHY_VERIFIED for WSB, but another problem is present (name it).
UNRESOLVED	Anything uncertain. Never forced into a class.

Never destroy a plant just to get a label
Splitting stems is only for plants the estate has already decided to remove. Healthy labels come from careful external inspection by the expert, recorded on the sheet.

5.4 Phone settings (do this once per phone)

- Use the **built-in camera app**. Do **not** use GPS Map Camera, Timestamp Camera or any app that writes text on the photo.
- iPhone: Settings > Camera > Formats > **Most Compatible** (saves JPEG instead of HEIC). Turn off Live Photos and portrait mode.
- Android: highest resolution, standard photo mode (not portrait/beauty/AI scene). Turn off watermark ("Shot on...").
- Keep **Location ON** in the camera: GPS in the hidden file data helps match photos to plants. We remove it during cleaning (Phase 3).
- Set the phone's date and time automatically. Timestamps are the backup for linking photos to sheets.
- No zoom beyond 2x; no flash unless the stem is in deep shade (note it on the sheet).

5.5 The photo sequence for every plant

Follow the same order every time. The ID card photo is what lets anyone, later, know which plant the next photos belong to.

#	Photo	How to take it
0	**ID card**	Card with estate code, plant ID, date, label (written after inspection) held in front of the stem. One quick photo.
1	**Whole plant**	From 2-3 m, whole plant in frame, camera at chest height.
2	**Main stem, front**	From 50-70 cm. Frame from ground level to the first primary branches. The stem should fill at least one-third of the image width. Sharp focus on the bark (tap to focus).
3	**Main stem, back**	Same as photo 2, from the opposite side of the stem.
4	**Close-up**	From 20-30 cm, of the most suspicious area (ridges/holes). For healthy plants, close-up of the same height on the side facing the path.
5	**Extra (optional)**	Additional symptoms, exit holes, frass, any other stress. For a split stem: the inside view (proof only).

Photograph before touching. For plants that will be split, take photos 0-4 first, then split and photograph the inside separately.

5.6 Good and bad examples from our own data



Good stem framing
stem fills frame, sharp, exit hole visible



Good close-up
exit hole clearly visible



Proof photo
split stem: tunnels + beetle (not for the field model)



Not usable as healthy
moss hides the bark: say so on the sheet



Not usable as healthy
canopy only, stem not visible



Weak
stem partly hidden by leaves, lime-coated

5.7 Quality checklist at capture time (5 seconds per photo)

- Is the main stem in focus? (Zoom in on the phone screen to check the bark texture.)
- Is the stem not blocked by leaves, hands or tools?
- Is the bark not burnt out to white by direct sun, nor black in deep shade? If so, move or shade with your body.
- Is the whole stem width visible, from ground level up?
- If any answer is "no", retake immediately. Retakes in the field cost seconds; later they are impossible.

5.8 Lighting

Do not try to make every photo identical. Instead, make sure **both classes are photographed in the same mix of light** (sun, partial shade, deep shade, overcast) and record the light on the sheet. The model must learn to ignore light, which it can only do if light is unrelated to the label.

5.9 The field sheet: add these columns

The current Varsapradaya Field Validation sheet is good. Add: **Plant ID** (matches the card), **Label** (Section 5.3), **Inspected by**, **Phone used**, **Photo numbers** (first and last), **Light**, **Moss/lichen cover** (none/some/heavy), **Lime-swabbed?** (yes/no). The full template is in Section 17.

5.10 End of every field day

1. Upload all original photos to `data/raw/YYYY-MM-DD_ESTATE/` (no edits, no WhatsApp compression; use a cable or Drive upload).
2. Photograph or scan the day's field sheets into the same folder.
3. Dataset lead types the sheets into plants.csv the same evening, while memories are fresh.
4. Run the cleaning and quality script (Phase 3) and send any "retake" list back to the field team.

Gate 1: pass before training
[ ] At least 100 HEALTHY_VERIFIED and 100 INFECTED (confirmed + expert-signed probable) plants
[ ] At least 4 estates, each with both classes; at least 3 phones, each used for both classes
[ ] Every plant has an ID-card photo, a main-stem photo and a sheet row signed by the expert
[ ] No overlay text on any photo; at least 90% of stem photos pass the quality checklist

6. Phase 2: Labelling and data management

6.1 Two levels of labels: keep them separate

Level	Example values	Who sets it
Plant label (ground truth)	INFECTED_CONFIRMED / INFECTED_PROBABLE / HEALTHY_VERIFIED / OTHER_STRESS / UNRESOLVED	Agronomist, on the sheet
Photo tags (what this photo shows)	view: id_card, whole_plant, stem_front, stem_back, closeup, split_inside visible: ridges, exit_hole, frass, moss, lime, none quality: good / retake	Dataset lead

A plant can be infected even if one of its photos shows nothing. The model learns from **plant labels**; photo tags decide which photos are used (e.g. only stem_front/stem_back/closeup with quality=good).

6.2 Folder structure

Recommended project layout

```
CWSB_project/
  data/
    raw/                      # originals exactly as from the phone (read-only, backed up)
      2026-11-20_EST01/       #   one folder per field day and estate
    clean/                    # output of Phase 3 (no GPS, no overlay, JPEG, upright)
    metadata/
      plants.csv              # one row per plant (from the field sheets)
      images.csv              # one row per photo
      estates.csv             # one row per estate
    splits/
      split_v1.csv            # image_id -> train / val / test / external_test
  src/                        # all scripts (Phases 3-8)
  models/                     # saved model files, one folder per version
  reports/                    # evaluation reports, error-review notes
  app/                        # API and phone app code
```

6.3 Linking photos to plants

1. Sort each day's photos by capture time.
2. Every ID-card photo starts a new plant; all photos until the next ID card belong to that plant.
3. The dataset lead opens each ID-card photo, reads the plant ID, and the script fills `plant_id` for the following photos.
4. Check: the number of plants in images.csv must equal the number of rows on that day's sheets.

Assign plant IDs from ID-card photos

```
import pandas as pd
imgs = pd.read_csv("data/metadata/images.csv").sort_values("capture_time")
current = None
for i, row in imgs.iterrows():
    if row["view"] == "id_card":
        current = row["plant_id_on_card"]        # typed by the dataset lead
    imgs.at[i, "plant_id"] = current
imgs.to_csv("data/metadata/images.csv", index=False)
```

6.4 Quality control of labels

- A second reviewer (or the agronomist on another day) re-checks a random 10% of plants, plus every INFECTED_PROBABLE.
- Disagreements go to UNRESOLVED until settled, never to a guess.
- Keep a `label_changes.csv` log: who changed what, when and why. Never overwrite silently.
- Freeze the dataset with a version name (dataset_v1.0) before training. Later additions become v1.1, v2.0 and so on.

Gate 2
[ ] Every photo has a plant_id; every plant has a signed label
[ ] 10% double-check done, with agreement of 90% or more
[ ] dataset_v1.0 frozen and backed up

7. Phase 3: Cleaning and preparing images

This step runs automatically on every new batch. It makes all images uniform and private without changing what is visible on the stem.

7.1 What the cleaning script does

Step	Why
Convert HEIC to JPEG; apply rotation from EXIF	One format, always upright
Remove all EXIF/XMP data (GPS, device serials)	Privacy; GPS is stored separately in estates.csv if needed
Detect and crop overlay text bands	Overlay text lets the model cheat; it must never reach training
Exact duplicates (SHA-256 hash) and near-duplicates (perceptual hash)	Copies inflate counts and leak between splits
Blur and exposure scores	Flag photos for retake; record scores for analysis
Keep full resolution (resize only when training)	Bark texture details matter

src/clean.py (core function)

```
import hashlib, pathlib, cv2, numpy as np
from PIL import Image, ImageOps
import pillow_heif
pillow_heif.register_heif_opener()

def clean_one(src: pathlib.Path, dst: pathlib.Path) -> dict:
    im = ImageOps.exif_transpose(Image.open(src)).convert("RGB")   # upright
    dst.parent.mkdir(parents=True, exist_ok=True)
    im.save(dst, "JPEG", quality=95, exif=b"")                     # no metadata at all
    g = cv2.imread(str(dst), cv2.IMREAD_GRAYSCALE)
    g = cv2.resize(g, (1024, int(1024 * g.shape[0] / g.shape[1])))
    return {
        "sha256": hashlib.sha256(src.read_bytes()).hexdigest(),
        "blur": cv2.Laplacian(g, cv2.CV_64F).var(),      # low = blurry
        "clipped": float((g > 250).mean()),              # share of burnt-out pixels
        "dark": float((g < 15).mean()),                  # share of black pixels
        "width": im.width, "height": im.height,
    }
```

Blur and exposure thresholds are not universal. Have the agronomist mark 50 photos as "usable" or "retake", then pick the blur and exposure values that best separate them.

7.2 Near-duplicate check

```
def dhash(path, size=8):
    g = Image.open(path).convert("L").resize((size + 1, size))
    a = np.asarray(g, dtype=np.int16)
    return (a[:, 1:] > a[:, :-1]).flatten()

def near_duplicates(paths, max_diff=6):
    h = {p: dhash(p) for p in paths}
    ps = list(h)
    return [(a, b) for i, a in enumerate(ps) for b in ps[i + 1:]
            if (h[a] != h[b]).sum() <= max_diff]       # review these pairs by eye
```

Near-duplicates of the **same plant** are fine (they stay together in one split). Near-duplicates across **different plant IDs** point to a labelling mistake and must be checked.

Gate 3
[ ] Every raw photo has a clean copy, or a recorded reason for exclusion
[ ] 0 files with EXIF/GPS in data/clean (checked by script)
[ ] Overlay check: a person has viewed the bottom strip of 50 random photos
[ ] Duplicate report reviewed

8. Phase 4: Splitting the data without cheating

The split decides whether your final number means anything. Two rules:

1. **All photos of one plant go to the same set.** (Group by plant_id.)
2. **At least one whole estate is kept completely aside as an external test.** It answers the real question: does the model work on a farm it has never seen?

Set	Content	Used for
external_test	1-2 entire estates (both classes), chosen before training	The final, honest result. Opened once.
test	About 15% of plants from the other estates	Final in-distribution result
val	About 15% of plants	Choosing epochs, settings and thresholds
train	About 70% of plants	Learning

src/make_split.py

```
import pandas as pd
from sklearn.model_selection import StratifiedGroupKFold

img = pd.read_csv("data/metadata/images.csv")
img = img[img.view.isin(["stem_front", "stem_back", "closeup"]) & (img.quality == "good")]
img = img[img.label.isin(["INFECTED_CONFIRMED", "INFECTED_PROBABLE", "HEALTHY_VERIFIED", "OTHER_STRESS"])]
img["y"] = img.label.str.startswith("INFECTED").astype(int)       # 1 = WSB, 0 = not WSB

EXTERNAL = ["EST07"]                                              # chosen in advance
img["split"] = "train"
img.loc[img.estate_id.isin(EXTERNAL), "split"] = "external_test"

rest = img[img.split == "train"]
sgkf = StratifiedGroupKFold(n_splits=7, shuffle=True, random_state=42)
folds = list(sgkf.split(rest, rest.y, groups=rest.plant_id))
img.loc[rest.index[folds[0][1]], "split"] = "test"   # ~1/7 of plants
img.loc[rest.index[folds[1][1]], "split"] = "val"    # ~1/7 of plants

# Leakage checks: must print empty sets
s = {k: set(img[img.split == k].plant_id) for k in img.split.unique()}
for a in s:
    for b in s:
        if a < b: print(a, b, s[a] & s[b])
img.to_csv("data/splits/split_v1.csv", index=False)
```

Report the split as a table of **plant counts** per set, per class, per estate and per phone. Every set must contain both classes.

> **Gate 4**
> [ ] Leakage check prints no overlaps
> [ ] Each set has both classes and at least 2 phones
> [ ] external_test estate(s) recorded in writing before any training

9. Phase 5: Training the model

We train in three steps, from simple to stronger. Each step is a useful result on its own, and the simple ones tell us quickly whether the data contains the signal at all.

Step	Method	Why
5A	Pretrained model as a fixed "feature extractor" + simple classifier (logistic regression)	Minutes to run, works with very little data, hard to overfit. Our first baseline.
5B	Fine-tune the pretrained model on stem photos	Usually better once there are a few hundred plants per class
5C (later)	Stem detector + classifier on the stem crop; or a hole/ridge detector	Focuses the model on the stem when photos are messy

9.1 Choosing the pretrained model

Model (timm name)	Size	Notes
vit_small_patch14_dinov2.lvd142m	22M	Excellent general features; best choice for step 5A
convnext_tiny.fb_in22k_ft_in1k	28M	Strong and easy to fine-tune; main choice for step 5B
efficientnet_b0.ra_in1k / mobilenetv3_large_100	5M / 5M	Small and fast; for running on the phone later
inception_v3 (the 2019 paper)	24M	Works, but older and weaker; only if you want to reproduce the paper

9.2 Step 5A: feature extractor + logistic regression

src/baseline_embed.py

```
import pandas as pd, numpy as np, torch, timm
from PIL import Image
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score

df = pd.read_csv("data/splits/split_v1.csv")
dev = "cuda" if torch.cuda.is_available() else "cpu"
m = timm.create_model("vit_small_patch14_dinov2.lvd142m", pretrained=True, num_classes=0).eval().to(dev)
cfg = timm.data.resolve_data_config({}, model=m)
tf = timm.data.create_transform(**cfg)

@torch.no_grad()
def embed(paths, bs=16):
    out = []
    for i in range(0, len(paths), bs):
        x = torch.stack([tf(Image.open(p).convert("RGB")) for p in paths[i:i + bs]]).to(dev)
        out.append(m(x).cpu().numpy())
    return np.concatenate(out)

X = {k: embed(df[df.split == k].path.tolist()) for k in ["train", "val"]}
y = {k: df[df.split == k].y.values for k in ["train", "val"]}
clf = LogisticRegression(C=0.5, max_iter=5000, class_weight="balanced").fit(X["train"], y["train"])
p = clf.predict_proba(X["val"])[:, 1]
print("val ROC-AUC", roc_auc_score(y["val"], p), " PR-AUC", average_precision_score(y["val"], p))
```

How to read it: PR-AUC near the share of positives means no signal; near 1.0 means strong signal. Also compute it **per plant** (average the scores of a plant's photos), which is how the app will be judged.

9.3 Step 5B: fine-tuning

Settings to start with (tune only on the validation set):

Setting	Starting value	Notes
Image size	384 x 384	Bark texture needs resolution; try 448 or 512 if GPU memory allows
Batch size	16	8 if out of memory
Optimizer	AdamW, weight decay 0.05	
Learning rate	1e-3 for the new head for 2 epochs (backbone frozen), then 1e-4 for all layers	Two-stage: warm up the head first
Epochs	20-40 with early stopping on val PR-AUC (patience 6)	Stop when validation stops improving
Loss	Cross-entropy with class weights (or a balanced sampler)	Handles unequal class sizes
Augmentation (train only)	Random resized crop (scale 0.6-1.0), horizontal flip, rotation of 10 degrees or less, brightness/contrast 20% or less, saturation 10% or less, slight blur sometimes	No vertical flips; no strong colour changes (yellowing is a clue)
Seed	Fixed (e.g. 42); run 3 seeds for the final report	Small datasets vary between runs

src/train.py (complete, minimal)

```
import torch, timm, pandas as pd, numpy as np
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms as T
from PIL import Image
from sklearn.metrics import average_precision_score

SIZE, BS, DEV = 384, 16, "cuda" if torch.cuda.is_available() else "cpu"
MEAN, STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)
train_tf = T.Compose([T.RandomResizedCrop(SIZE, scale=(0.6, 1.0), ratio=(0.6, 1.4)),
                      T.RandomHorizontalFlip(), T.RandomRotation(10),
                      T.ColorJitter(0.2, 0.2, 0.1, 0.02),
                      T.RandomApply([T.GaussianBlur(5)], p=0.15),
                      T.ToTensor(), T.Normalize(MEAN, STD)])
eval_tf = T.Compose([T.Resize(SIZE), T.CenterCrop(SIZE), T.ToTensor(), T.Normalize(MEAN, STD)])

class Stems(Dataset):
    def __init__(self, df, tf): self.df, self.tf = df.reset_index(drop=True), tf
    def __len__(self): return len(self.df)
    def __getitem__(self, i):
        r = self.df.iloc[i]
        return self.tf(Image.open(r.path).convert("RGB")), int(r.y)

df = pd.read_csv("data/splits/split_v1.csv")
tr, va = df[df.split == "train"], df[df.split == "val"]
dl_tr = DataLoader(Stems(tr, train_tf), BS, shuffle=True, num_workers=2)
dl_va = DataLoader(Stems(va, eval_tf), BS, num_workers=2)

model = timm.create_model("convnext_tiny.fb_in22k_ft_in1k", pretrained=True,
                          num_classes=2, drop_path_rate=0.1).to(DEV)
w = torch.tensor([1.0, (tr.y == 0).sum() / max((tr.y == 1).sum(), 1)], dtype=torch.float, device=DEV)
loss_fn = torch.nn.CrossEntropyLoss(weight=w)

def run_epoch(opt=None):
    model.train(opt is not None); ys, ps = [], []
```

(continued)

```
    for x, y in (dl_tr if opt else dl_va):
        x, y = x.to(DEV), y.to(DEV)
        with torch.set_grad_enabled(opt is not None):
            out = model(x); loss = loss_fn(out, y)
            if opt: opt.zero_grad(); loss.backward(); opt.step()
        ys += y.tolist(); ps += out.softmax(1)[:, 1].tolist()
    return average_precision_score(ys, ps)

# Stage 1: train only the new head
for p in model.parameters(): p.requires_grad = False
for p in model.get_classifier().parameters(): p.requires_grad = True
opt = torch.optim.AdamW(model.get_classifier().parameters(), lr=1e-3, weight_decay=0.05)
for e in range(2): run_epoch(opt)

# Stage 2: fine-tune everything with a small learning rate
for p in model.parameters(): p.requires_grad = True
opt = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=0.05)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=40)
best, bad = 0, 0
for e in range(40):
    run_epoch(opt); sched.step(); ap = run_epoch()
    print(f"epoch {e}  val PR-AUC {ap:.3f}")
    if ap > best: best, bad = ap, 0; torch.save(model.state_dict(), "models/v1/best.pt")
    else:
        bad += 1
        if bad >= 6: break
```

Common beginner mistakes
Augmenting the validation or test photos.
Looking at the test set while choosing settings. Use val only.
Reporting training accuracy. Only val/test/external numbers count.
Mixing whole-plant, stem and split-stem photos in one class without thinking. V1 trains on stem views only.
Comparing to the 2019 paper's "85.5%". Different data and a different split, so it is not comparable.

9.4 Step 5C (later): focus on the stem

If photos often contain lots of leaves and background, add a first stage that finds the stem:

1. Draw a box around the main stem on about 300 photos in CVAT or Label Studio (about 2 hours of work).
2. Train a small detector (e.g. YOLO from the `ultralytics` package) to find the stem.
3. Crop the stem with some margin and give the crop to the classifier from step 5B.
4. Optional: label exit holes and ridges with boxes and train a detector that points to the exact damage. This makes results explainable to farmers.

Gate 5
[ ] Step 5A baseline done and recorded
[ ] Step 5B val PR-AUC better than 5A (or a reason why not)
[ ] Model file, settings, seed and dataset version saved together in models/v1/

10. Phase 6: Evaluation: proving it works

10.1 Turn photo scores into plant decisions

The app judges a plant, not a photo. Average the WSB scores of the plant's stem photos (or take the highest score, which is more cautious), then apply the thresholds.

10.2 Choose thresholds on validation, then freeze them

Threshold selection (validation only)

```
import numpy as np
from sklearn.metrics import precision_recall_curve

# plant-level scores and labels from the VALIDATION set
prec, rec, thr = precision_recall_curve(y_val_plant, score_val_plant)
# "WSB likely" cut-off: the highest threshold that still gives recall >= 0.90
ok = np.where(rec[:-1] >= 0.90)[0]
T_HIGH = thr[ok].max()
# "no visible signs" cut-off: below this, at most 2% of infected val plants remain
T_LOW = np.quantile(score_val_plant[y_val_plant == 1], 0.02)
print("WSB likely if score >=", T_HIGH, "; healthy if score <", T_LOW, "; otherwise inconclusive")
```

10.3 What to report

Report	Content
Overall (test and external_test separately)	Confusion matrix in plants; recall, precision, PR-AUC, inconclusive rate; 95% confidence interval (bootstrap over plants)
Per estate	The same numbers per estate; the external estate is the headline result
Per phone	Detects "this phone confuses the model"
Per light condition	Sun / partial shade / shade / overcast
Per condition	Moss-covered, lime-swabbed, other-stress: false-alarm rate
Per severity	Recall for early (ridges only) vs advanced infestations
Heat maps (Grad-CAM)	For about 30 photos: is the model looking at the stem and holes, or at leaves/background?

Grad-CAM check

```
from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
cam = GradCAM(model=model, target_layers=[model.stages[-1]])        # ConvNeXt last stage
heat = cam(input_tensor=x.unsqueeze(0))[0]                           # x = eval-transformed photo
overlay = show_cam_on_image(rgb_float_0_to_1, heat, use_rgb=True)    # save and review by eye
```

10.4 Error-review meeting (with the agronomist)

- Look at **every missed infected plant** and **every false alarm** on validation.
- For each: was the label right? Was the stem visible? Light, phone, moss? Early infestation?
- Wrong label: fix it in the log. Bad photo: improve the capture guide. A real model weakness: collect more of that case.
- Never delete hard cases from the test set to improve the number.

10.5 Acceptance criteria for a field pilot (to be confirmed by the team)

Metric (external estate, plant level)	Suggested minimum
Recall on INFECTED_CONFIRMED	90% or more
Precision (share of alerts that are real)	70% or more
Inconclusive rate on good-quality photos	15% or less
False-alarm rate on mossy/lime-swabbed healthy stems	No more than 1.5x the overall false-alarm rate
Grad-CAM	Model attends to the stem in most reviewed cases

Gate 6
[ ] Thresholds chosen on val and written down before opening test/external_test
[ ] External estate results meet the criteria, or the gap is understood and a plan is agreed
[ ] Error review done and recorded

11. Phase 7: Improving the model

11.1 Learning curve: do we need more data?

Train on 25%, 50%, 75% and 100% of the training **plants** and plot validation PR-AUC. If it is still rising at 100%, more plants will help, so collect more. If it is flat, improve the photos, labels or method instead.

11.2 What to try, in order of usual impact

1. Fix labels found wrong in error review.
2. Collect more of the failing case (e.g. early infestation, mossy healthy stems, a specific phone).
3. Higher image resolution (448 or 512), or stem cropping (step 5C).
4. A different backbone (ConvNeXt vs EfficientNet vs DINOv2 fine-tune).
5. Test-time augmentation: average the score over the photo and its mirror image.
6. Ensembling 3 seeds or 2 architectures.

Change one thing at a time, and record every run (dataset version, settings, val result) in a run log. MLflow or a simple spreadsheet both work.

12. Phase 8: Deployment: API and phone app

12.1 Architecture (start simple: prediction on a server)

Part	What it does
Phone app	Guides the photos (overlay frame for the stem), checks blur/brightness on the phone, uploads
API (FastAPI)	Receives photos, runs the same cleaning as training, runs the model, applies the thresholds
Storage	Saves photos and results with the model version (private, access-controlled)
Expert review queue	Photos flagged "WSB likely" or "inconclusive" can be reviewed by an expert; their answer becomes new training data

app/api.py (sketch)

```
import io, torch, timm
from fastapi import FastAPI, UploadFile, File
from PIL import Image, ImageOps

app = FastAPI()
MODEL_VERSION, T_LOW, T_HIGH = "cwsb_v1", 0.30, 0.60      # from Phase 6, frozen
model = timm.create_model("convnext_tiny.fb_in22k_ft_in1k", num_classes=2)
model.load_state_dict(torch.load("models/v1/best.pt", map_location="cpu")); model.eval()

@app.post("/v1/cwsb/predict")
async def predict(stem_front: UploadFile = File(...), stem_back: UploadFile | None = File(None)):
    scores = []
    for f in [stem_front, stem_back]:
        if f is None: continue
        im = ImageOps.exif_transpose(Image.open(io.BytesIO(await f.read()))).convert("RGB")
        # quality_check(im) -> return "inconclusive" with a reason if blurry / dark / burnt out
        with torch.no_grad():
            scores.append(model(eval_tf(im).unsqueeze(0)).softmax(1)[0, 1].item())
    s = max(scores)
    status = "wsb_likely" if s >= T_HIGH else "no_visible_signs" if s < T_LOW else "inconclusive"
    return {"status": status, "model_version": MODEL_VERSION}
```

12.2 What the farmer sees

- Never show the raw score as a percentage unless it has been calibrated and validated. Use the three messages from Section 2.
- Always add: "This is a screening tool. Confirm with an expert before any treatment."
- For "inconclusive", say why ("photo blurry", "stem not visible") and how to retake.
- Work offline-first: queue uploads when there is no network (common in estates).

12.3 Running on the phone (later, optional)

After the server version is validated, export a small model (EfficientNet-B0 / MobileNetV3) to ONNX or TFLite and check that the phone gives the **same answers** as the server on 200 test photos before switching.

Export to ONNX

```
dummy = torch.randn(1, 3, 384, 384)
torch.onnx.export(model, dummy, "models/v1/cwsb_v1.onnx", input_names=["image"],
                  output_names=["logits"], opset_version=17)
```

Gate 8
[ ] API returns identical scores to the evaluation script on 50 photos
[ ] Pilot with 5-10 users on 2 estates, with an expert checking every "WSB likely"
[ ] Clear limitation text approved by the agronomist

13. Phase 9: Monitoring and retraining

- **Dashboard (weekly):** number of checks, share of each answer, inconclusive reasons, results by phone/estate, expert-confirmed outcomes.
- **Warning signs:** inconclusive rate rising (new phones or new photo habits), alerts mostly rejected by experts (false alarms), or confirmed cases that the model missed.
- **Retraining loop:** expert-confirmed field cases go into the next dataset version, then retrain, compare on the **same frozen external test** plus a new estate, and release only if better. Keep the old model for rollback.
- **Season check:** re-evaluate after each flight period. Symptoms look different right after emergence.

14. Optional: adding the environmental and sensor data

The team already records air temperature, humidity, light (lux), stem temperature, soil moisture, pH, Brix, NDVI and shade on the hotspot sheets. That is valuable, but it answers a **different question** from the photos:

	Image model (this guide)	Risk model (future)
Question	Does this plant show WSB signs now?	Is this block likely to see infestation soon?
Input	Stem photos	Weather, shade, sensors, soil, history
Unit	Plant	Block / estate, per week or season
Data needed	Hundreds of verified plants	Several seasons of records with infestation counts

Recommendation: keep recording the sensor data with the same plant and block IDs from day one, so it can be joined later. Build the risk model after 1-2 seasons of data. The 2019 paper's weather models failed mainly because of too little data, which confirms this. Early analysis can still look for simple patterns, e.g. infestation rate vs shade level, which the team notes already suggest ("study the shade patterns"; "irrigating during flight period, less incidence").

15. Timeline, gates and roles

An example schedule starting mid-November 2026. Collection dates depend on estate access and expert availability.

Weeks	Work	Gate
1	Team, decisions, permissions, phones, field kits, sheet update (Phase 0)	G0
2-7	Field collection during winter flight period (Phase 1); daily ingestion and cleaning (Phase 3) in parallel	G1
3-8	Linking photos to plants, labels, double-checks (Phase 2)	G2, G3
8	Split and leakage checks (Phase 4)	G4
8-9	Baseline 5A, then fine-tuning 5B (Phase 5)	G5
10-11	Evaluation, thresholds, error review; learning curve (Phases 6-7)	G6
11-12	Fill gaps: extra collection if needed; retrain	
13-16	API + app prototype; restricted pilot with expert checking (Phase 8)	G8
Apr-May 2027	Summer flight: second collection round, model v2, wider pilot	

If time is short, protect this
Never shorten the external-estate test or skip expert verification to meet a date. A model that looks good on paper and fails on farms damages trust with growers.

16. Risks and how to handle them

Risk	Effect	What we do
Too few infected plants found	Model cannot learn	Collect in flight season; ask CCRI / Coffee Board / estates for known infested blocks; ask the 2019 and 2023 authors for data
Healthy and infected from different estates/phones	Model learns site/phone, not pest	Pair every infected plant with nearby healthy plants, same phone (Section 5.2)
Labels wrong	Model learns noise	Expert signs every plant; second check; UNRESOLVED instead of guessing
Moss, lichen, lime confuse the model	False alarms	Deliberately include such healthy stems; report their false-alarm rate
Early infestation invisible	Misses	App says "no visible signs", not "healthy"; encourage repeat checks in flight season
Overlay apps, compression	Cheating / lost detail	Normal camera only; upload originals; cleaning script checks
Farmers treat output as diagnosis	Harmful decisions	Careful wording; expert referral; no treatment advice in V1
Data privacy / ownership	Legal and trust issues	Written consent; GPS removed from images; access control

17. Templates and checklists

17.1 plants.csv: one row per plant

```
plant_id,estate_id,block,date,label,diagnosis_basis,inspected_by,phone,first_photo,last_photo,light,moss_cover,lime_swabb
EST01-P001,EST01,B3,2026-11-20,INFECTED_CONFIRMED,exit_holes+frass,AGRO1,iPhone13,IMG_2001,IMG_2005,partial_shade,some,no
EST01-P002,EST01,B3,2026-11-20,HEALTHY_VERIFIED,full_stem_inspection,AGRO1,iPhone13,IMG_2006,IMG_2010,partial_shade,heavy
```

17.2 images.csv: one row per photo

```
image_id,plant_id,estate_id,raw_path,clean_path,view,quality,visible,phone,capture_time,blur,clipped,dark,sha256
IMG_2001,EST01-P001,EST01,raw/2026-11-20_EST01/IMG_2001.JPG,clean/IMG_2001.jpg,id_card,good,,iPhone13,2026-11-20 10:02:11
IMG_2003,EST01-P001,EST01,raw/2026-11-20_EST01/IMG_2003.JPG,clean/IMG_2003.jpg,stem_front,good,ridges;exit_hole,iPhone13,
```

17.3 Field kit checklist

- Charged phones (3+ models) + power bank; normal camera app only; Location ON; date/time automatic
- Plant-ID cards or a small whiteboard + marker; numbered tags/ribbons
- Printed field sheets (with the new columns) + clipboard + pens
- Sensor kit as already used (thermometer, lux meter, soil probe, etc.)
- Gloves; secateurs/saw only for plants already marked for removal

17.4 Daily capture card (print and laminate)

Per plant: 0 ID card · 1 whole plant (2-3 m) · 2 stem front (50-70 cm, ground to first branches) · 3 stem back · 4 close-up (20-30 cm) · 5 extras
Before leaving the plant: in focus? stem unblocked? not burnt out or black? label + photo numbers written on the sheet?
Both classes at every estate · every phone for both classes · include mossy and lime-coated healthy stems

17.5 Final sign-off checklist before a pilot

- [ ] dataset_v1 frozen; plant counts per class, estate and phone documented
- [ ] Every training plant has an expert-signed label
- [ ] Split by plant; external estate untouched until the final test
- [ ] Thresholds set on val; external results meet the criteria
- [ ] Error review and Grad-CAM review recorded
- [ ] App wording approved; inconclusive path works; expert referral in place
- [ ] Model, data and threshold versions recorded together; rollback possible

18. What we kept from the paper and the ChatGPT plan, and what we changed

Source	Kept	Changed and why
2019 ISPRS paper (Bhandarkar et al.)	Transfer learning from a pretrained CNN; uncontrolled field photos; the idea of a phone app; WSB biology and symptom list	Main input is stem photos, not whole plants. Its accuracy is not a target: the reported 84-87% does not match its own confusion matrix (70%), and class counts do not add up (1,600 vs 2,425). Weather modelling postponed: the paper's attempt failed for lack of data.
ChatGPT handbook	Data audit first; plant-level labels; split by plant and estate; "inconclusive" output; cautious farmer messaging; gates; versioning	Based on our audit: overlay-camera ban, ID-card photo protocol, paired healthy plants at every estate, mossy/lime healthy stems, stem-first capture, flight-season timing. Modern backbones (DINOv2 / ConvNeXt) instead of Inception-v3, with a quick feature-extractor baseline first.

19. Glossary

Term	Meaning
Augmentation	Random, realistic changes to training photos (crop, rotate, brightness) so the model generalises.
Backbone	The pretrained part of the model that turns a photo into features.
Calibration	Checking that a score of 0.8 really means about 80% of such plants are infected.
Confusion matrix	Table of TP, FP, FN, TN.
Data leakage	Information from test photos sneaking into training (e.g. same plant in both). Inflates results.
Epoch	One pass over all training photos.
Exit hole	Oval hole left when an adult beetle leaves the stem. Proof that the plant was infested.
External test	Evaluation on estates the model has never seen. The most honest result.
Fine-tuning	Continuing to train a pretrained model on our photos with a small learning rate.
Flight period	When adult beetles emerge: Oct–Dec (winter) and Apr–May (summer).
Frass	Powdery wood waste pushed out by larvae.
Grad-CAM	A heat map showing which part of the photo drove the model's answer.
Ground truth	The best verified answer for a plant (expert inspection).
Overfitting	The model memorises training photos and fails on new ones.
PR-AUC	One number (0-1) summarising precision and recall over all thresholds. Good for unequal classes.
Precision / Recall	Share of alerts that are real / share of infected plants that are caught.
Ridges	Raised rings or bands on the bark caused by larvae feeding under it.
Threshold	Score cut-off that turns a 0-1 score into a decision.
Transfer learning	Starting from a model pretrained on millions of images and adapting it to our task.
Validation set	Photos used to choose settings and thresholds; never used for learning.